Guía de Actividades Práctico-Experimentales Nro. 006

1. Datos Generales

Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	6
Título de la Práctica	Mensajería Distribuida con RabbitMQ y MQTT. PARTE 2: MICROSERVICIOS CON RABBITMQ Y SPRING BOOT
Nombre del Docente	José O. Guamán Q.
Fecha	27 de octubre de 2026 30 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Implementar microservicios en Spring Boot con Spring AMQP que consuman las colas de RabbitMQ generadas en la Parte 1, configuren exchanges y bindings, reenvíen alertas a una cola de notificaciones y expongan una API REST para consultar el estado de la flota.

- Comprender los fundamentos del protocolo MQTT y su aplicacion en entornos IoT
- Configurar e instalar un broker Mosquitto para mensajeria publicador/suscriptor
- Implementar publicadores y suscriptores MQTT en Python usando paho-mqtt
- Disenar topic hierarchies para organizar la telemetria de una flota vehicular
- Entender el protocolo AMQP 0-9-1 y el modelo de intercambio de RabbitMQ
- Configurar exchanges, colas y bindings en RabbitMQ

- Implementar microservicios en Spring Boot que consuman mensajes de RabbitMQ
- Integrar MQTT y RabbitMQ mediante un servicio puente en Python
- Aplicar patrones de mensajeria: pub/sub, routing y competing consumers.

3. Materiales y reactivos:

Guia practica

Software

- Python
- paho-mqtt
- pika
- Java JDK 17
- Spring Boot
- Docker
- Docker Compose
- Mosquito
- RabbitMQ
- SQLite3
- IDE
- Postman
- Navegador web

Hardware

- Computador

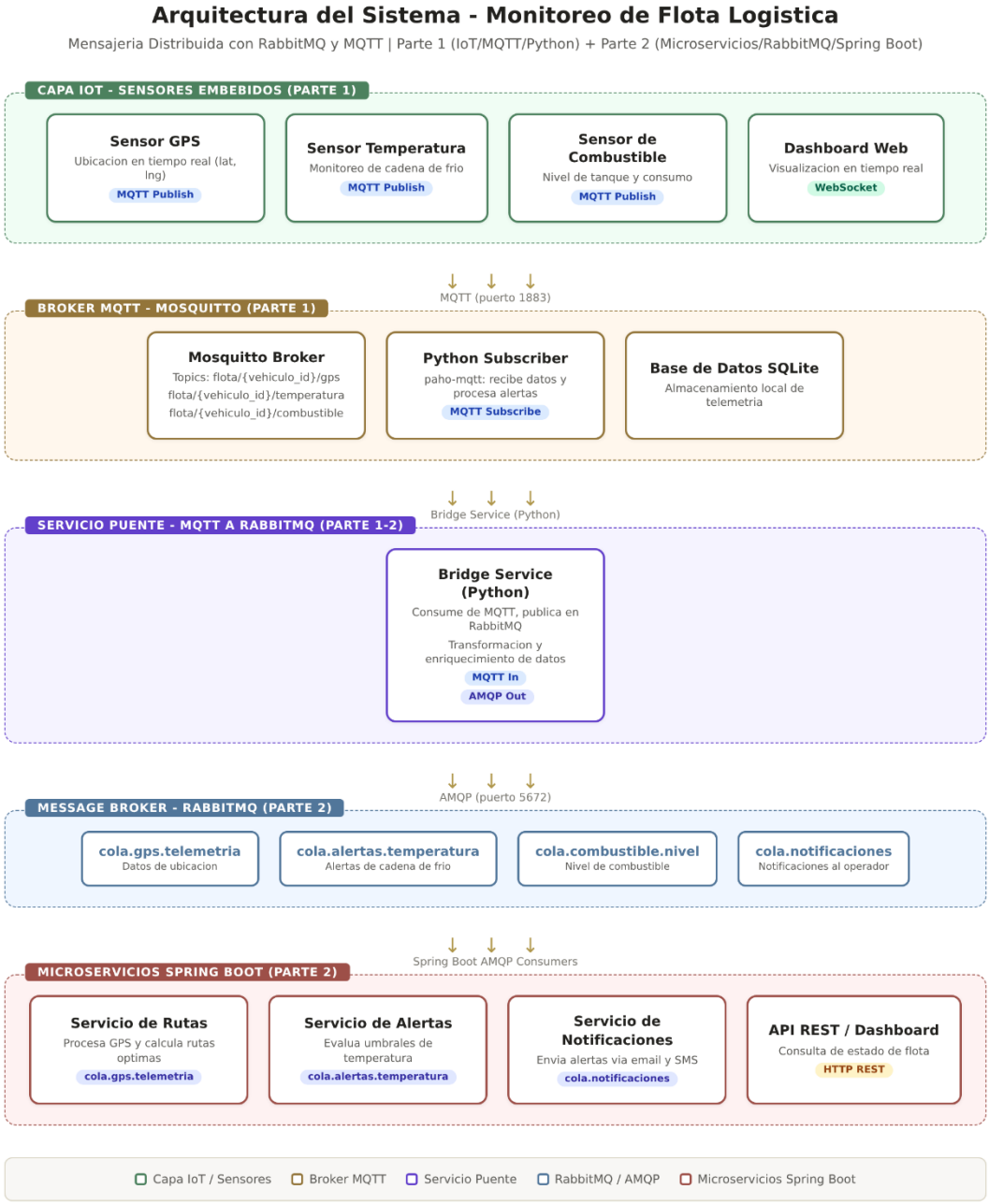
4. Metodología/procedimiento:

En el contexto de esta práctica, trabajaremos con dos protocolos de mensajeria ampliamente utilizados en la industria: MQTT (Message Queuing Telemetry Transport) y RabbitMQ (basado en el protocolo AMQP). Aunque ambos facilitan la comunicacion asincrona entre componentes, sus diseños responden a necesidades muy diferentes. MQTT es un protocolo ligero diseñado específicamente para entornos IoT (Internet de las Cosas). Opera sobre el patrón publicador/suscriptor, donde los dispositivos publican mensajes en topics jerarquicos y los suscriptores reciben los mensajes de los topics a los que están suscritos. Su diseño prioriza el mínimo consumo de ancho de banda, lo que lo ideal para dispositivos con recursos limitados como sensores, microcontroladores y módulos GPS. MQTT define tres niveles de QoS (Quality of Service): 0 (como máximo una vez, sin garantía), 1 (al menos una vez, con retransmisión), y 2 (exactamente una vez, con handshake de cuatro pasos). RabbitMQ es un broker de mensajeria empresarial que implementa el protocolo AMQP 0-9-1 (Advanced Message Queuing Protocol). A diferencia de MQTT, RabbitMQ opera con un modelo mas sofisticado que incluye exchanges, colas, bindings y routing keys. Este modelo permite implementar multiples patrones de mensajeria como point-to-point, pub/sub, routing y topic-based routing. RabbitMQ ofrece caracteristicas empresariales como persistencia de mensajes, confirmaciones de entrega, colas duraderas, y una interfaz de administración web. El escenario de esta practica simula un Sistema de Monitoreo de Flota Logistica: una empresa de logistica necesita

monitorear en tiempo real su flota de vehiculos de reparto. Cada vehiculo esta equipado con sensores IoT que envian datos de telemetria: posicion GPS, temperatura del compartimento de carga (cadena de frio) y nivel de combustible. La arquitectura propuesta combina ambos protocolos: la capa IoT (sensores en vehiculos) utiliza MQTT para publicar datos de telemetria de forma eficiente y ligera, mientras que la capa de microservicios internos utiliza RabbitMQ para el procesamiento confiable de alertas, almacenamiento y notificaciones.

PARTE 2: MICROSERVICIOS CON RABBITMQ Y SPRING BOOT.

En esta segunda parte (3 horas), implementaremos los microservicios de procesamiento que consumen los mensajes de RabbitMQ. Utilizaremos Spring Boot con Spring AMQP para crear consumidores que procesen los datos de telemetría y las alertas, y expondremos una API REST para consultar el estado de la flota.



Configuracion del Entorno

Cree un archivo docker-compose separado para RabbitMQ. A diferencia de Mosquitto, RabbitMQ incluye una interfaz de administracion web en el puerto 15672 que permite visualizar colas, exchanges, conexiones y mensajes en tiempo real.

docker-compose-rabbitmq.yml

version: '3.8'

services:

rabbitmq:

image: rabbitmq:3-management

container_name: rabbitmq_broker

ports:
- "5672:5672"
- "15672:15672"
environment:
RABBITMQ_DEFAULT_USER: admin
RABBITMQ_DEFAULT_PASS: admin123
restart: unless-stopped

Inicie RabbitMQ con docker-compose -f docker-compose-rabbitmq.yml up -d. Acceda a la interfaz de administracion en http://localhost:15672 con usuario admin y contrasena admin123.

Actividad 5: Crear Proyecto Spring Boot (30 min)

Utilice Spring Initializr (start.spring.io) para crear el proyecto con las siguientes dependencias:
spring-boot-starter-amqp, spring-boot-starter-web, spring-boot-starter-data-jpa y H2 Database. El grupo sera com.fleet.monitor y el artefacto fleet-monitor.
Recuerde utilice jdk 17 y este enlace: https://start.spring.io

En java cree RabbitMQConfig.java

```
package com.fleet.monitor.config;
import org.springframework.amqp.core.*;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
@Configuration
public class RabbitMQConfig {
public static final String GPS_QUEUE =
"cola.gps.telemetria";
public static final String TEMP_ALERT_QUEUE =
"cola.alertas.temperatura";
public static final String FUEL_QUEUE =
"cola.combustible.nivel";
public static final String NOTIFICATION_QUEUE =
"cola.notificaciones";
public static final String FLEET_EXCHANGE =
"exchange.fleet";
@Bean
public DirectExchange fleetExchange() {
return new DirectExchange(FLEET_EXCHANGE);
}
@Bean
public Queue gpsQueue() {
return new Queue(GPS_QUEUE, true);
}
```

```
@Bean
public Queue tempAlertQueue() {
return new Queue(TEMP_ALERT_QUEUE, true);
}
@Bean
public Queue fuelQueue() {
return new Queue(FUEL_QUEUE, true);
}
@Bean
public Queue notificationQueue() {
return new Queue(NOTIFICATION_QUEUE, true);
}
@Bean
public Binding gpsBinding(Queue gpsQueue,
DirectExchange fleetExchange) {
return BindingBuilder.bind(gpsQueue)
.to(fleetExchange).with("gps.routing");
}
@Bean
public Binding tempBinding(Queue tempAlertQueue,
DirectExchange fleetExchange) {
return BindingBuilder.bind(tempAlertQueue)
.to(fleetExchange).with("temp.alert");
}
@Bean
public Binding fuelBinding(Queue fuelQueue,
DirectExchange fleetExchange) {
return BindingBuilder.bind(fuelQueue)
.to(fleetExchange).with("fuel.routing");
}
}
```

Actividad 6: Servicio de Telemetria GPS (45 min)

Crearemos un consumidor RabbitMQ que procese los mensajes de GPS y los almacene. También definiremos el modelo de datos y un controlador REST para exponer la telemetría.

GpsMessage.java (Modelo)

```
package com.fleet.monitor.model;
public class GpsMessage {
private String vehicleId;
private String timestamp;
```

```
private double lat;
private double lng;
private double speed;
// Getters y setters
public String getVehicleId() { return vehicleId; }
public void setVehicleId(String vehicleId) {
this.vehicleId = vehicleId; }
public String getTimestamp() { return timestamp; }
public void setTimestamp(String timestamp) {
this.timestamp = timestamp; }
public double getLat() { return lat; }
public void setLat(double lat) { this.lat = lat; }
public double getLng() { return lng; }
public void setLng(double lng) { this.lng = lng; }
public double getSpeed() { return speed; }
public void setSpeed(double speed) {
this.speed = speed; }
}
```

GpsConsumer.java (Consumidor)

```
package com.fleet.monitor.consumer;
import com.fleet.monitor.config.RabbitMQConfig;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.annotation
.RabbitListener;
import org.springframework.stereotype.Component;
@Component
public class GpsConsumer {
private static final Logger log =
LoggerFactory.getLogger(GpsConsumer.class);
@RabbitListener(
queues = RabbitMQConfig.GPS_QUEUE)
public void consumeGps(String message) {
try {
log.info("GPS recibido: {}", message);
// Almacenar en base de datos
} catch (Exception e) {
log.error("Error procesando GPS: {}",
e.getMessage());
}
}
```

```
}
```

Actividad 7: Servicio de Alertas (45 min)

El servicio de alertas consume mensajes de las colas de temperatura y combustible, y los reenvia a una cola de notificaciones centralizada. Este patron de enrutamiento demuestra como RabbitMQ permite la transformacion y redistribucion de mensajes entre microservicios.

Archivo: AlertConsumer.java

```
package com.fleet.monitor.consumer;
import com.fleet.monitor.config.RabbitMQConfig;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.annotation
.RabbitListener;
import org.springframework.amqp.rabbit.core
.RabbitTemplate;
import org.springframework.stereotype.Component;
@Component
public class AlertConsumer {
private static final Logger log =
LoggerFactory.getLogger(AlertConsumer.class);
private final RabbitTemplate rabbitTemplate;
public AlertConsumer(
RabbitTemplate rabbitTemplate) {
this.rabbitTemplate = rabbitTemplate;
}
@RabbitListener(
queues = RabbitMQConfig.TEMP_ALERT_QUEUE)
public void consumeTempAlert(String message) {
log.warn("ALERTA TEMPERATURA: {}", message);
rabbitTemplate.convertAndSend(
RabbitMQConfig.NOTIFICATION_QUEUE,
message);
}
@RabbitListener(
queues = RabbitMQConfig.FUEL_QUEUE)
public void consumeFuelAlert(String message) {
log.warn("ALERTA COMBUSTIBLE: {}", message);
rabbitTemplate.convertAndSend(
RabbitMQConfig.NOTIFICATION_QUEUE,
message);
}
```

```
}
```

Actividad 8: API REST de Consulta (30 min)

Finalmente, crearemos un controlador REST que permita consultar el estado de la flota. Esta API expone endpoints para obtener un resumen general del estado de la flota y para consultar la telemetría individual de cada vehículo.

FleetController.java

```
package com.fleet.monitor.controller;
import org.springframework.web.bind.annotation.*;
import org.springframework.http.ResponseEntity;
import java.util.*;
@RestController
@RequestMapping("/api/fleet")
public class FleetController {
@GetMapping("/status")
public ResponseEntity<Map<String, Object>>
getFleetStatus() {
Map<String, Object> status = new HashMap<>();
status.put("totalVehicles", 3);
status.put("activeAlerts", 2);
status.put("timestamp", new Date());
return ResponseEntity.ok(status);
}
@GetMapping("/vehicle/{id}/telemetria")
public ResponseEntity<List<String>>
getTelemetria(@PathVariable String id) {
return ResponseEntity.ok(
Collections.singletonList(
"Datos de telemetria para " + id));
}
}
```

Pruebe la API con el siguiente comando o con Postman:

```
curl http://localhost:8080/api/fleet/status
curl http://localhost:8080/api/fleet/vehicle/VH-001/telemetria
```

5. Resultados esperados:

- Microservicios Spring Boot: Una aplicación Spring Boot con consumidores RabbitMQ que procesan datos de GPS y alertas, y una API REST para consultar el estado de la flota.

- Integracion end-to-end: Un flujo de datos completo donde los datos de sensores viajan desde el simulador a traves de MQTT, el puente, RabbitMQ, los microservicios, hasta la API REST).

6. Preguntas de Control:

- Explique los cuatro tipos de exchanges en RabbitMQ (direct, fanout, topic, headers) y proporcione un ejemplo de uso en el contexto de la flota logistica?
- Que es una cola durable en RabbitMQ y por que es importante? Que sucederia si el broker se reinicia y las colas no son durables?
- Compare el patron pub/sub con el patron point-to-point. Cuales son las ventajas de pub/sub para el monitoreo de flotas?
- Que medidas de seguridad se pueden implementar en MQTT y RabbitMQ? Incluya autenticacion, autorizacion, cifrado TLS y control de acceso

7. Evaluación

Criterio	Excelente	Bueno	Bajo
Funcionamiento (back-end, front-end) se evaluara criterios de Verificacion	Funciona completo (3)	Parcial (2)	No funciona (0)
Documento de practica (Enlace a git, metodología usada por el alumno, resultados, preguntas de control, bibliografía, conclusiones, video de explicación)	Funciona completo (3)	Parcial (2)	No funciona (0)
Defensa de practica	Todos los puntos (4)	Parcial (1)	No valido (0)

En el video se debe explicar de forma clara y completa todos los puntos del documento, incluyendo las soluciones, servicios levantados como MQTT y rabbitQ, metodologías y resultados. Además, se deben responder las preguntas de control y presentar conclusiones, asegurando que toda la solución quede bien comprendida

8. Bibliografía

Semana 5. Guaman, Jose. 2026

Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	Validar únicamente en FirmaEC. Firmado electrónicamente por: JOSE OSWALDO GUAMAN QUINCHE
Ejecutado Por	Rene Rivera **Técnico Docente**	Validar únicamente en FirmaEC. Firmado electrónicamente por: RAMIRO RENE RIVERA GUAMAN
Aprobado por	Edison L Coronel Romero **Director de Carrera**	Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMERO Validar únicamente con FirmaEC